

TAMALE TECHNICAL UNIVERSITY
FACULTY OF APPLIED SCIENCE AND TECHNOLOGY
COMPUTER SCIENCE DEPARTMENT

**TODOFLOW: DESIGN AND DEVELOPMENT OF AN OFFLINE-
FIRST
WEB-BASED TASK MANAGEMENT SYSTEM FOR
TAMALE TECHNICAL UNIVERSITY**

BY

[STUDENT FULL NAME 1] — [MATRICULATION NUMBER]

[STUDENT FULL NAME 2] — [MATRICULATION NUMBER]

[STUDENT FULL NAME 3] — [MATRICULATION NUMBER]

A PROJECT SUBMITTED TO THE DEPARTMENT OF COMPUTER SCIENCE,
TAMALE TECHNICAL UNIVERSITY, IN PARTIAL FULFILMENT OF THE
REQUIREMENTS FOR THE AWARD OF HIGHER NATIONAL DIPLOMA IN
INFORMATION AND COMMUNICATION TECHNOLOGY

[MONTH, 2026]

DECLARATION

We hereby declare that this submission is our own work and that, to the best of our knowledge and belief, it contains no material previously published or written by another person, nor material which to a substantial extent has been accepted for the award of any other diploma or degree at Tamale Technical University, or any other educational institution, except where due acknowledgement has been made in the text.

Name	Matriculation Number	Signature	Date
[STUDENT FULL NAME 1]	[MATRIC NO.]		
[STUDENT FULL NAME 2]	[MATRIC NO.]		
[STUDENT FULL NAME 3]	[MATRIC NO.]		

CERTIFICATION

We hereby certify that the preparation and presentation of this project work were supervised in accordance with the guidelines on project work approved by the Computer Science Department, Tamale Technical University.

Supervisor: [SUPERVISOR FULL NAME]

Signature:

Date:

Head of Department: [HOD FULL NAME]

Signature:

Date:

External Assessor:

Signature:

Date:

ACKNOWLEDGEMENT

We wish to express our sincere gratitude to our supervisor, [SUPERVISOR FULL NAME], for the guidance, patience, and constructive criticism offered throughout the design and development of this project. Their willingness to review our work at every stage, from the initial problem statement to the final testing phase, shaped this report into what it has become.

We are also grateful to the entire staff of the Computer Science Department, Tamale Technical University, for the technical foundation laid over the course of our study, and to the Faculty of Applied Science and Technology for providing an environment that encouraged practical, hands-on learning.

Finally, we thank our families, classmates, and everyone who supported, tested, and gave feedback on TodoFlow during its development. Their patience with early, unfinished versions of the system helped us build a better final product.

DEDICATION

This project is dedicated to our parents and families, whose sacrifices made our education possible, and to every student of Tamale Technical University who has ever tried to keep track of coursework, deadlines, and project milestones using nothing more than memory and scattered paper notes. We hope TodoFlow makes that a little easier for the next person.

ABSTRACT

Task and time management remain persistent challenges for students, small teams, and individual professionals across Ghana, many of whom still rely on paper notebooks, memory, or fragmented chat messages to track what needs to be done. This project presents the design, development, and testing of TodoFlow, a web-based, offline-first task management system built to address this gap for students and staff of Tamale Technical University and similarly situated users elsewhere in Ghana and the wider West African sub-region, where mobile data affordability and network reliability are recurring constraints on software adoption (Shanahan & Bahia, 2024).

TodoFlow was built using the Laravel 13 PHP framework with the Livewire 4 full-stack reactive component library and Bootstrap 5 for the user interface, backed by a relational database (SQLite in development, MySQL in production). The system allows a user to organise tasks into lists, tag them for cross-cutting categorisation, set due dates and reminders, and track task status through a simple todo–doing–done workflow. Deleted items are soft-deleted into a recoverable Trash rather than destroyed immediately, and a scheduled background job generates in-app alerts for overdue tasks, tasks due today, and passed reminders. An administrative panel allows a designated administrator to manage user accounts across the system.

Architecturally, the application separates presentation (Livewire components), business logic (a dedicated service layer), and data (Eloquent models and policies), with a shared error-handling trait ensuring that a failure in any single action is caught, logged to a dedicated log channel, and reported to the user as a toast notification rather than crashing the interface. The system was tested using an automated suite of over one hundred feature tests covering authentication, authorisation, and every create/read/update/delete workflow in the application, alongside manual browser-based testing of the complete user interface. The resulting system demonstrates that a modern, resilient, and genuinely usable task management tool can be built and deployed using free, open-source technologies well suited to the infrastructure realities of a Ghanaian tertiary institution.

TABLE OF CONTENTS

DECLARATION	ii
CERTIFICATION	iii
ACKNOWLEDGEMENT	iv
DEDICATION	v
ABSTRACT	vi
TABLE OF CONTENTS	vii
LIST OF TABLES	x
LIST OF FIGURES	xi
LIST OF ABBREVIATIONS	xii
CHAPTER ONE: PROJECT OVERVIEW	1
1.1 Introduction	1
1.2 Problem Statement	1
1.3 Objectives of Project	2
1.3.1 General Objective.....	2
1.3.2 Specific Objectives.....	2
1.4 Significance of Project	2
1.5 Scope of Project	3
1.6 Organization of Document.....	3
CHAPTER TWO: SYSTEM AND LITERATURE REVIEW.....	5
2.1 Introduction	5
2.2 Review of Existing Systems	5
2.2.1 Case Study 1: Todoist	5
2.2.2 Case Study 2: Microsoft To Do	5
2.2.3 Case Study 3: Google Tasks	6
2.2.4 Drawbacks of Reviewed Systems	6
2.2.5 Comparison	6

2.3 Review of Literature	7
CHAPTER THREE: METHODOLOGY	9
3.1 Introduction	9
3.2 System Development	9
3.2.1 Methodology Used	9
3.2.2 Frontend Programming Languages	10
3.2.3 Backend Development Tools	11
3.3 Proposed System	11
3.3.1 Description	11
3.3.2 Advantages of Proposed System	12
3.3.3 System Specification	13
3.3.3.1 Functional Requirements	13
3.3.3.2 Non-Functional Requirements	13
3.3.3.3 Hardware Requirements	14
3.3.3.4 Software Requirements	14
3.4 Module Description	14
3.5 Design Notations	15
3.5.1 Data Flow Diagram	15
3.5.2 Use Case Diagram	16
3.5.3 ER Diagram	17
3.5.4 System Architecture Diagram	18
3.5.5 Database Schema	19
CHAPTER FOUR: TESTING AND IMPLEMENTATION	21
4.1 Introduction	21
4.2 System Testing	21
4.2.1 Alpha Testing	21
4.2.2 Beta Testing	23

4.3 System Implementation.....	23
CHAPTER FIVE: SUMMARY AND RECOMMENDATIONS.....	26
5.1 Introduction	26
5.2 Summary	26
5.3 Recommendations	26
5.3.1 Recommendations for Future Developers.....	26
5.3.2 Recommendations for End Users.....	27
5.3.3 Recommendations for Future Research	27
REFERENCES.....	29
APPENDIX.....	30
A.1 Screen Shots	30

LIST OF TABLES

Note: the caption numbers in this list resolve correctly once Word updates the field below (this is a normal one-time step for any Word document with an auto-generated table). In Word: click anywhere inside the list, press Ctrl+A, then F9 — or right-click and choose “Update Field”.

Table : Comparison of Reviewed Task Management Systems	6
---	----------

LIST OF FIGURES

Note: the caption numbers in this list resolve correctly once Word updates the field below (this is a normal one-time step for any Word document with an auto-generated table). In Word: click anywhere inside the list, press Ctrl+A, then F9 — or right-click and choose “Update Field”.

<i>Fig : The Iterative Development Model Adopted for TodoFlow.....</i>	<i>10</i>
<i>Fig : Context-Level Data Flow Diagram (Level 0)</i>	<i>15</i>
<i>Fig : Level 1 Data Flow Diagram.....</i>	<i>16</i>
<i>Fig : Use Case Diagram for User and Administrator Actors.....</i>	<i>17</i>
<i>Fig : Entity Relationship Diagram of the TodoFlow Database</i>	<i>18</i>
<i>Fig : Layered System Architecture of the TodoFlow Application.....</i>	<i>19</i>
<i>Fig : Database Schema Showing Tables and Columns</i>	<i>20</i>
<i>Fig : TodoFlow Dashboard — Todo List View with Filters</i>	<i>24</i>
<i>Fig : Administrator Overview Screen</i>	<i>25</i>
<i>Fig : Login Screen.....</i>	<i>30</i>
<i>Fig : Registration Screen</i>	<i>31</i>
<i>Fig : Tag Management Screen, Showing System and Personal Tags.....</i>	<i>31</i>
<i>Fig : Trash Screen with Model-Type Filter</i>	<i>31</i>
<i>Fig : User Profile Screen</i>	<i>32</i>
<i>Fig : Administrator User Management Screen.....</i>	<i>32</i>

LIST OF ABBREVIATIONS

Abbreviation	Meaning
HND	Higher National Diploma
TTU	Tamale Technical University
UI	User Interface
UX	User Experience
CRUD	Create, Read, Update, Delete
DFD	Data Flow Diagram
ERD	Entity Relationship Diagram
DBMS	Database Management System
SQL	Structured Query Language
PHP	Hypertext Preprocessor (recursive acronym)
HTML	HyperText Markup Language
CSS	Cascading Style Sheets
JSON	JavaScript Object Notation
API	Application Programming Interface
MVC	Model-View-Controller
URL	Uniform Resource Locator
UUID	Universally Unique Identifier
CI	Continuous Integration
SME	Small and Medium-sized Enterprise
GSMA	GSM Association (Global System for Mobile Communications Association)

CHAPTER ONE: PROJECT OVERVIEW

1.1 Introduction

Every student, small business owner, and working professional eventually runs into the same quiet problem: too many things to remember and no reliable place to put them. At Tamale Technical University, as at most tertiary institutions in Ghana, this shows up in familiar ways — an assignment deadline missed because it was written on a torn page that got lost, a group project task nobody remembers agreeing to, a lecturer's reminder buried three hundred messages deep in a class WhatsApp group. None of these are failures of intelligence or effort; they are failures of tooling.

TodoFlow is our response to that gap. It is a web-based task management system that lets a user create todos, organise them into lists, tag them, set due dates and reminders, and track progress through a simple workflow, all wrapped in an interface that works whether the user's connection is fast, slow, or briefly gone. This chapter introduces the problem TodoFlow addresses, the objectives that guided its development, and the scope within which it was built.

1.2 Problem Statement

Students and staff of Tamale Technical University, like much of the wider Ghanaian and West African market, currently coordinate personal and team tasks using an inconsistent mix of paper notebooks, phone “Notes” apps, and chat platforms never designed for task tracking. None of these tools distinguish between a task that is merely noted and one that is actually due; none of them remind a user before a deadline passes; and none of them survive the intermittent network conditions that remain common outside Ghana's major urban centres, where mobile broadband coverage exists but affordability and consistent usage remain real barriers (Shanahan & Bahia, 2024). The result is a pattern familiar to almost every student: work that was fully intended to be done, forgotten simply because the system for remembering it was inadequate. TodoFlow was conceived to solve this specific problem — not by inventing a new category of software, but by building a task manager that is fast, clear, resilient to network interruptions, and genuinely pleasant enough to use every day.

1.3 Objectives of Project

1.3.1 General Objective

The general objective of this project is to design, develop, and test a web-based, offline-tolerant task management system that allows individuals and small teams to create, organise, and track tasks reliably, with particular attention to the usability and connectivity constraints typical of a Ghanaian tertiary institution.

1.3.2 Specific Objectives

The specific objectives of this project are to:

1. Design a relational database schema capable of representing users, todos, todo lists, tags, and their relationships, including support for soft-deletion and future multi-device synchronisation.
2. Develop a Livewire-based web application implementing full create, read, update, and delete functionality for todos, todo lists, and tags, with per-user authorisation enforced at every layer.
3. Implement a recoverable Trash system and an in-app alert mechanism that notifies a user of overdue tasks, tasks due today, and passed reminders.
4. Implement an administrative panel through which a designated administrator can view system-wide statistics and manage user accounts.
5. Test the completed system through automated feature tests and manual browser-based testing, and document the development process.

1.4 Significance of Project

TodoFlow is useful to at least three distinct groups. First, students at Tamale Technical University and similar institutions gain a single, reliable place to track coursework, project milestones, and personal commitments, reducing the missed deadlines that come from relying on memory or scattered notes. Second, small teams and student groups working on shared projects — including future HND project groups — benefit from shared todo lists and tags that make dividing and tracking group work easier than a chat thread ever could. Third, small business owners and freelance professionals in Ghana's growing digital economy, who often cannot justify the cost of enterprise project-management software, gain access to a free, open-source alternative built with their connectivity realities in mind. Beyond these immediate beneficiaries, this project also serves the Computer Science Department

itself: as a documented, fully-tested Laravel and Livewire application, TodoFlow is intended to stand as a reference point for future HND project groups exploring similar web-based system development.

1.5 Scope of Project

This project covers the design, development, and testing of the TodoFlow web application: user registration and authentication; full CRUD management of todos, todo lists, and tags, including a system-level tag concept shared read-only across all users; a recoverable Trash for todos, lists, tags, and registered devices; an in-app alert centre driven by a scheduled background command; a user profile area with avatar upload, password change, and theme preference; and an administrative panel for user account management. The project also includes the underlying database schema and service-layer architecture that make the application maintainable and resilient to runtime errors. It was developed and demonstrated on a local development server; full production deployment, real-time multi-device synchronisation, and native mobile packaging are identified as boundaries of the current work, discussed further as recommendations in Chapter Five. Within these boundaries, an administrator can manage user accounts and view aggregate statistics, but does not have direct access to an individual user's private todo content — a deliberate privacy boundary maintained throughout the system's authorisation policies.

1.6 Organization of Document

This project work is presented in five chapters. Chapter One introduces the project, covering the background to the study, the problem statement, the objectives, the significance of the work, and its scope. Chapter Two reviews existing task management systems similar to TodoFlow, alongside published literature relevant to offline-first software design and task management productivity. Chapter Three describes the methodology adopted for the project and details how the system was designed, including its data flow, use case, entity relationship, and architectural diagrams. Chapter Four presents how the system was tested and implemented, including the challenges encountered during development. Chapter Five summarises the work completed and offers recommendations for future developers, end users, and further research.

CHAPTER TWO: SYSTEM AND LITERATURE REVIEW

2.1 Introduction

Before designing TodoFlow, it was necessary to understand how existing task management systems approach the same problem, and what published research says about task management software and offline-first design more broadly. This chapter reviews three widely-used existing systems, identifies their general drawbacks relative to the needs of a Ghanaian tertiary-institution user base, and summarises relevant literature that informed the architectural decisions made in Chapter Three.

2.2 Review of Existing Systems

Three widely-adopted task management applications were reviewed as points of comparison for TodoFlow: Todoist, Microsoft To Do, and Google Tasks. Each was selected because it targets the same broad use case as TodoFlow — personal and small-team task tracking — while representing a different vendor ecosystem and design philosophy.

2.2.1 Case Study 1: *Todoist*

Todoist, developed by Doist, is one of the longest-running dedicated task management applications still in active use, having grown out of a personal project by its founder in the mid-2000s into a commercial product used by individuals and organisations worldwide. It offers projects, labels (tags), priority levels, natural-language due date entry, and a paid "Karma" gamification layer intended to encourage consistent use. Todoist is widely regarded as one of the more mature products in this space, with a large individual user base and a smaller but meaningful set of business/team subscribers. Its main limitations, from the perspective of this project's target users, are that its most useful features (unlimited reminders, more than five active projects, and detailed activity history) sit behind a recurring paid subscription that is difficult for a Ghanaian student to justify or pay for using locally available payment methods.

2.2.2 Case Study 2: *Microsoft To Do*

Microsoft To Do is Microsoft's free task management application, built on the technology it acquired with Wunderlist in 2015 and re-launched under its current name in 2017. It integrates tightly with Outlook and the wider Microsoft 365

ecosystem, offering lists, steps (sub-tasks), due dates, and a daily "My Day" planning view. Its core functionality is free, which makes it more accessible than Todoist's paid tiers. However, it is deliberately built as a companion to Outlook and Microsoft 365 rather than a standalone product, so features such as tagging, custom workflows, and system-wide default categories shared across a user base are either missing or awkward to use outside that ecosystem, and full functionality assumes a Microsoft account and a broadly reliable connection to Microsoft's cloud services.

2.2.3 Case Study 3: Google Tasks

Google Tasks began as a lightweight feature embedded in Gmail before Google released it as a standalone mobile application in 2018. Its strength is deep integration with Gmail and Google Calendar, letting a user turn an email into a task in a couple of clicks. Its feature set is intentionally minimal: simple lists, due dates, and sub-tasks, with no tagging, no priority levels, and no shared or system-level categorisation. This simplicity is also its main limitation for a project-tracking or team-coordination use case — Google Tasks was never designed to be a full task management system, and it shows in the absence of any organisational structure beyond a flat list.

2.2.4 Drawbacks of Reviewed Systems

Across all three systems, three recurring drawbacks stood out relative to the needs identified in Chapter One. First, none of them treat intermittent connectivity as a first-class design concern; each assumes a broadly reliable connection to a vendor's cloud service, with offline support offered only as a secondary caching layer rather than a core design principle. Second, meaningful organisational features (unlimited projects, tagging, activity history) are frequently paywalled, which is a real barrier for students without access to international payment cards. Third, none of the three offer any concept of a shared, administrator-managed baseline (equivalent to TodoFlow's system-level tags) that an institution could roll out consistently to every user without each person configuring their own categories from scratch.

2.2.5 Comparison

Table 1: Comparison of Reviewed Task Management Systems

Feature	Todoist	Microsoft To Do	Google Tasks	TodoFlow

Free to use	Partial (paid tiers)	Yes	Yes	Yes
Tagging / labels	Yes (paid limits)	No	No	Yes
Shared system-wide categories	No	No	No	Yes
Recoverable trash / soft delete	Limited	Limited	No	Yes
Built-in alert centre	Reminders only	Reminders only	No	Yes
Admin/institutional account management	Business tier only	Via Microsoft 365 admin	Via Google Workspace admin	Yes (built-in)
Open-source / self-hostable	No	No	No	Yes

2.3 Review of Literature

Beyond comparable products, three published sources directly informed the design decisions behind TodoFlow.

Pothineni (2024) examines offline-first mobile architecture as a design paradigm in which network access is treated as an optional enhancement rather than a strict requirement for an application to function. The paper discusses synchronisation strategies, local data persistence, and the trade-offs between consistency and availability that arise when a client can operate without a live connection to its backend. This directly informed TodoFlow's decision to soft-delete records instead of destroying them immediately, and its broader database design (UUIDs, version numbers, and client-timestamp columns present on every core model) that anticipates a future offline-sync layer even though full multi-device sync was placed outside this project's immediate scope, as noted in section 1.5.

Shanahan and Bahia (2024), in the GSMA's State of Mobile Internet Connectivity report, document that while mobile broadband coverage continues to expand across Sub-Saharan Africa, a substantial "usage gap" persists — many people within reach of mobile internet infrastructure still do not use it consistently, owing

chiefly to affordability and digital literacy barriers rather than a lack of signal. This finding grounded the problem statement in section 1.2 and reinforced the decision to build a lightweight, server-rendered application (Livewire, rather than a heavier client-side JavaScript framework) that keeps data transfer per interaction small.

Nurzi and Wahab (2022), in their study of a web-based student task management system built to support remote group coursework during the disruption of the COVID-19 pandemic, found that giving students a shared, structured system for tracking group tasks measurably improved coordination and task completion compared to informal chat-based coordination. This finding directly supports the significance argued for in section 1.4 — that a purpose-built task tracker, even a relatively simple one, outperforms informal tools for group and academic task coordination — and shaped TodoFlow's decision to support shared, system-level tags rather than only fully private, per-user categorisation.

CHAPTER THREE: METHODOLOGY

3.1 Introduction

This chapter describes how TodoFlow was built. It begins with the development methodology adopted for the project and the reasoning behind it, followed by the specific frontend and backend technologies used and why each was chosen over the available alternatives. It then presents the proposed system in detail: its functional, non-functional, hardware, and software requirements, followed by a description of the system's major modules. The chapter closes with the design notations produced during development — the data flow diagrams, use case diagram, entity relationship diagram, system architecture diagram, and database schema that together document how TodoFlow's parts fit together. Read alongside Chapter Four, this chapter is intended to let another developer understand not just what TodoFlow does, but how and why it was built the way it was.

3.2 System Development

3.2.1 *Methodology Used*

TodoFlow was developed using an iterative and incremental (agile-influenced) development model rather than a strict waterfall approach. Each feature area of the system — core todo management, tag management, the trash/soft-delete system, the alert system, and the admin panel — was planned, designed, implemented, and tested as its own short cycle, with working, testable software produced at the end of every cycle rather than only at the very end of the project. This was chosen over a waterfall model because the exact shape of several features (particularly the alert system and the service-layer refactoring described in section 4.2.1) only became clear once earlier features were already working, and an iterative model made it possible to revisit and improve earlier decisions without treating the whole project as a single monolithic phase. Each iteration followed the same six-step cycle shown in Figure 3.1.

Iterative & Incremental (Agile) Development Model

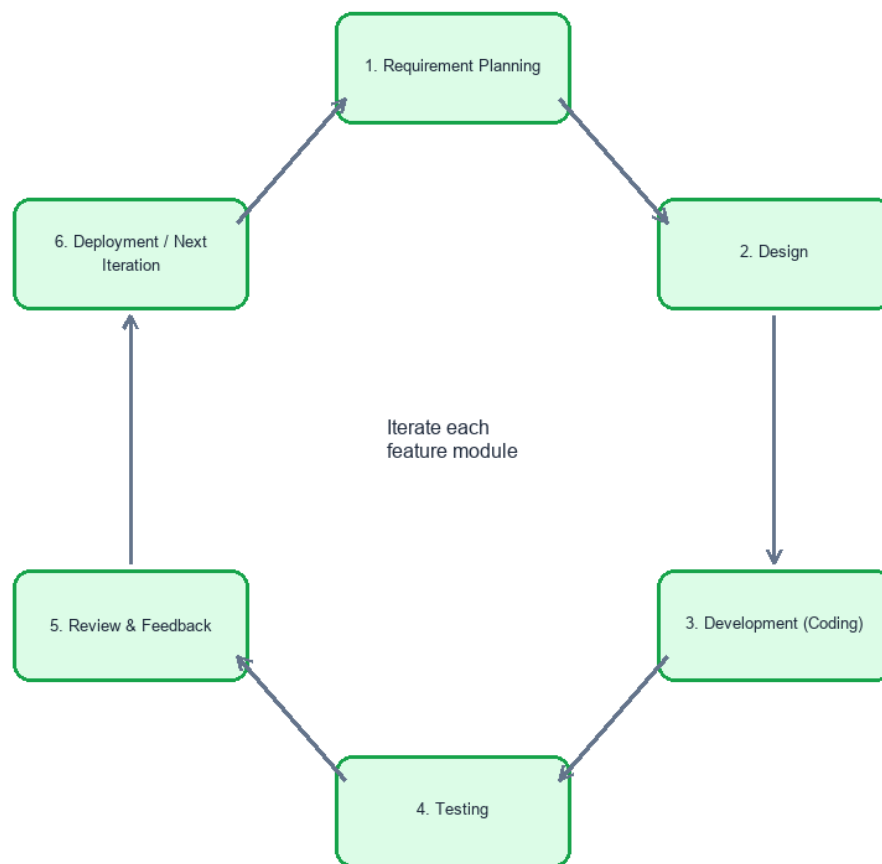


Figure 3.7 (Fig. in Chapter 3): The Iterative Model Adopted for TodoFlow Development

Fig 1: The Iterative Development Model Adopted for TodoFlow

3.2.2 Frontend Programming Languages

TodoFlow's frontend is built primarily with Blade, Laravel's server-side templating language, combined with Livewire 4 and Bootstrap 5. Blade compiles PHP templates into plain PHP for fast rendering and supports template inheritance and component includes, which kept the view layer organised as the application grew. Livewire is a full-stack framework that allows a Blade component's PHP class to be updated reactively from the browser over AJAX requests without hand-written JavaScript for most interactions — clicking a button, submitting a form, or toggling a todo's status all trigger a small server round-trip that re-renders only the affected part of the page. This was chosen deliberately over a separate JavaScript single-page framework such as React or Vue because it lets a small team write one language (PHP) end to end, and because Livewire's small, targeted payloads are better suited

to the variable-bandwidth conditions described in section 1.2 than a heavier client-rendered JavaScript bundle. Bootstrap 5 supplies the visual component library — buttons, modals, dropdowns, form controls — on top of which TodoFlow's own design system (custom CSS variables for light and dark themes) was layered. Minimal vanilla JavaScript was used only where no server round-trip was appropriate, such as toast notifications and the custom confirmation dialog.

Livewire was first released in 2020 by Caleb Porzio and has since become one of the most widely adopted packages in the Laravel ecosystem; version 4, used in this project, introduced improved lifecycle hooks and JavaScript interoperability that made it practical to integrate small hand-written behaviours (such as the toast and confirm-dialog systems) alongside server-driven components without conflict.

3.2.3 Backend Development Tools

The backend is built on Laravel 13, a PHP web application framework, running on PHP 8.4. Laravel was chosen for its mature ecosystem (Eloquent ORM, a built-in authorisation policy system, database migrations, queued jobs, and a task scheduler), all of which TodoFlow uses directly. During development, the application served requests using PHP's built-in development server and stored data in SQLite, a lightweight, file-based, zero-configuration relational database engine well suited to local development and automated testing; in a production deployment the same Eloquent-based codebase runs unchanged against MySQL, a widely available open-source relational database management system supported by essentially every Ghanaian shared-hosting and cloud provider. Laravel itself was first released by Taylor Otwell in 2011 and has grown into the most widely used PHP framework worldwide; it was chosen over writing plain PHP or using a lighter micro-framework because its conventions (migrations, policies, the scheduler used for TodoFlow's alert generation, and its testing tools) removed the need to hand-build infrastructure that was needed anyway, letting development time go into TodoFlow's actual features rather than framework plumbing.

3.3 Proposed System

3.3.1 Description

TodoFlow is a web application that lets an authenticated user manage tasks ("todos") through their web browser. A user signs in and lands on a dashboard

showing their todos, grouped by an optional list and filterable by priority, tag, and free-text search. Each todo carries a title, an optional description, a priority level, an optional due date and reminder, and any number of tags. A todo progresses through a simple status workflow — todo, doing, done — toggled with a single click, and can be marked a favourite for quick access. Deleting a todo, a list, or a tag does not destroy it immediately; it moves to a Trash screen from which the user can restore it or delete it permanently. A bell icon in the header opens an alert centre listing todos that are overdue, due today, or whose reminder time has passed, generated automatically every five minutes by a scheduled background command. A profile page lets the user update their name, email, avatar, timezone, and password, and toggle between light and dark themes. A separate administrative area, visible only to accounts flagged as administrators, shows system-wide statistics and lets an administrator search, deactivate, and restore user accounts.

3.3.2 Advantages of Proposed System

Because most students and small teams at Tamale Technical University currently have no dedicated task management tool at all — relying instead on paper, memory, and general-purpose chat apps — TodoFlow's first advantage is simply that it exists as a purpose-built alternative. Beyond that baseline, it offers several concrete advantages over both the informal tools it replaces and the existing systems reviewed in Chapter Two.

First, every destructive action in TodoFlow is recoverable by design: todos, lists, tags, and devices are soft-deleted into a Trash rather than removed outright, which none of the three systems reviewed in section 2.2 guarantee consistently. Second, TodoFlow's system-level tags give every user immediate access to a shared, administrator-curated set of categories (such as "urgent" or "work") without any setup, addressing the gap identified in section 2.2.4. Third, because the entire codebase is open-source and built on freely available tools, it can be self-hosted by the department at no licensing cost, unlike the paid tiers of Todoist or the vendor-account dependence of Microsoft To Do and Google Tasks. Fourth, its error-handling architecture (described fully in section 4.2.1) means a failure in one action is caught, logged, and reported to the user as a message rather than crashing the whole interface — an operational reliability property not typically discussed in commercial products of this kind.

3.3.3 System Specification

3.3.3.1 Functional Requirements

A functional requirement describes a specific behaviour the system must perform — a task it must be able to carry out in response to a user action. TodoFlow's main functional requirements are:

- The system shall allow a visitor to register a new account and log in with an email and password.
- The system shall allow an authenticated user to create, edit, complete, and delete todos, organised into lists and tags.
- The system shall soft-delete records and allow a user to restore or permanently delete them from a Trash view.
- The system shall generate and display in-app alerts for overdue, due-today, and reminder-triggered todos.
- The system shall allow an administrator to view system statistics and manage (deactivate/restore) user accounts.

3.3.3.2 Non-Functional Requirements

A non-functional requirement describes a quality the system must exhibit while performing its functions, rather than a specific function itself. TodoFlow's main non-functional requirements are:

- Usability — the interface must remain clear and operable on both desktop and mobile screen sizes.
- Reliability — an unexpected error in any single user action must not crash the application for that user or any other.
- Security — a user must never be able to view, modify, or delete another user's data, enforced through authorisation policies checked on every action.
- Performance — routine interactions (adding a todo, toggling its status) should complete in well under one second on a typical shared-hosting server.
- Maintainability — business logic must be isolated in a dedicated service layer, not scattered across the interface code, so that future developers can extend one feature without risking another.

3.3.3.3 Hardware Requirements

The minimum hardware needed to run and use TodoFlow is modest, reflecting its web-based nature:

- Server: any machine capable of running PHP 8.2 or later with at least 512MB of available RAM.
- Client: any desktop, laptop, tablet, or smartphone capable of running a modern web browser.
- Network: an internet or local network connection sufficient to load a typical web page (a broadband connection is not required).

3.3.3.4 Software Requirements

The following software was required to build and run TodoFlow:

- PHP 8.4.18
- Laravel Framework 13
- Livewire 4.3
- Bootstrap 5 and Bootstrap Icons
- SQLite (development) / MySQL 8 (production)
- Composer 2 (PHP dependency manager) and Node.js with npm (frontend asset build)
- Pest 4 (automated testing framework) and Laravel Pint (code-style enforcement)

3.4 Module Description

TodoFlow is organised into six major modules. The Authentication module handles registration, login, and logout. The Todo module is the core of the system, covering creation, editing, status/favourite toggling, reordering, and deletion of todos, and the tag-resolution logic that reuses an existing tag by name rather than creating duplicates. The List and Tag Management module lets a user organise todos into named lists and reusable tags, including administrator-owned system tags visible to every user. The Trash module presents every soft-deleted todo, list, tag, and device in one place with restore and permanent-delete actions. The Alert module runs a scheduled command every five minutes that scans for overdue, due-today, and reminder-triggered todos and generates in-app notifications, displayed through a bell-icon dropdown in the header. The Administration module, accessible only to

administrator accounts, presents system-wide statistics and a searchable, paginated list of users that an administrator can deactivate, restore, or edit.

3.5 Design Notations

3.5.1 Data Flow Diagram

Figure 3.2 presents the Level 0 (context) data flow diagram, showing TodoFlow as a single process exchanging data with its two external entities — the ordinary User and the Administrator — and with the Task Scheduler that drives alert generation. Figure 3.3 decomposes this into a Level 1 diagram, breaking the system into its six major processes (authentication, todo management, tag/list management, trash management, alert generation, and user administration) and the four data stores each process reads from or writes to.

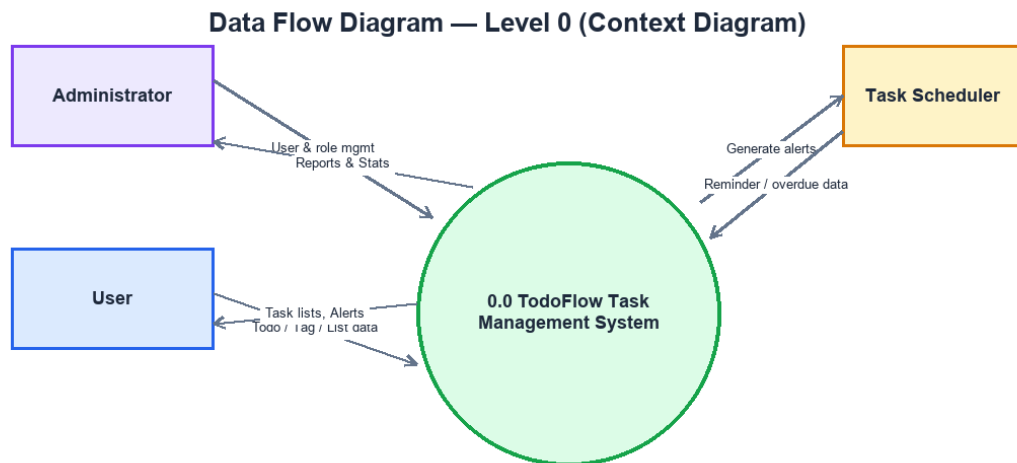


Figure 3.2: Context-Level Data Flow Diagram (Level 0)

Fig 2: Context-Level Data Flow Diagram (Level 0)

Data Flow Diagram — Level 1

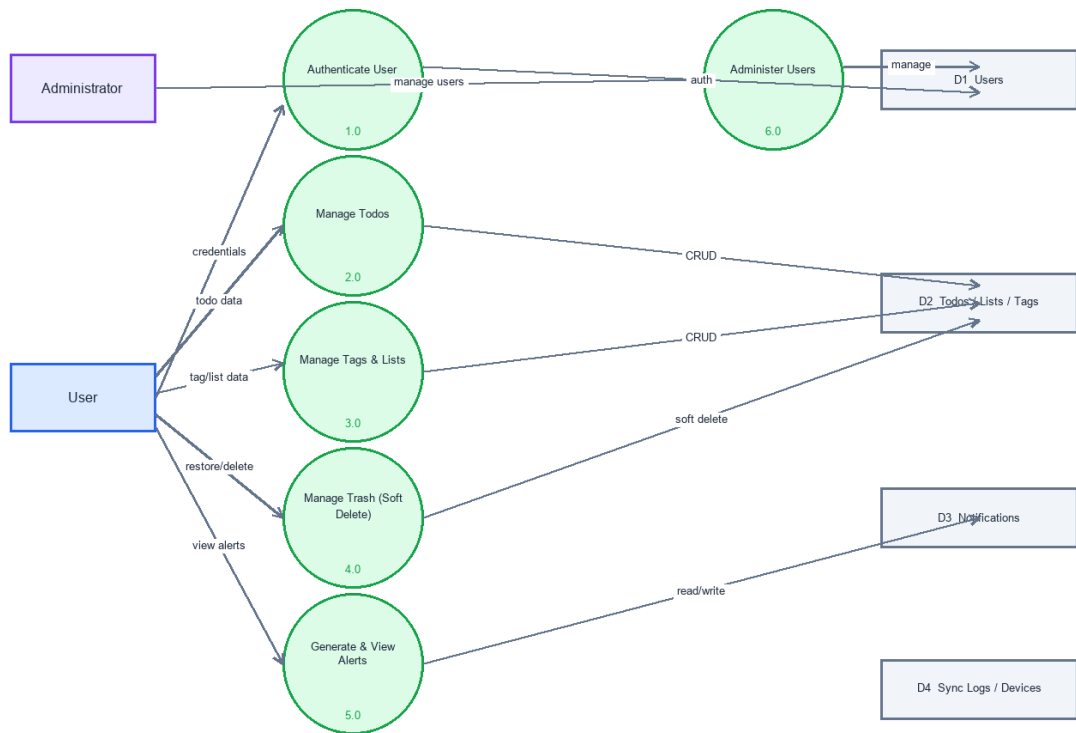


Figure 3.3: Level 1 Data Flow Diagram showing major processes and data stores

Fig 3: Level 1 Data Flow Diagram

3.5.2 Use Case Diagram

Figure 3.4 shows the use case diagram for TodoFlow's two actors. A regular User can register, authenticate, and fully manage their own todos, lists, tags, trash, alerts, profile, and devices. An Administrator inherits every use case available to a regular User (an administrator is, after all, still a user of their own todos) and additionally has access to user account management and the system overview.

Use Case Diagram — TodoFlow

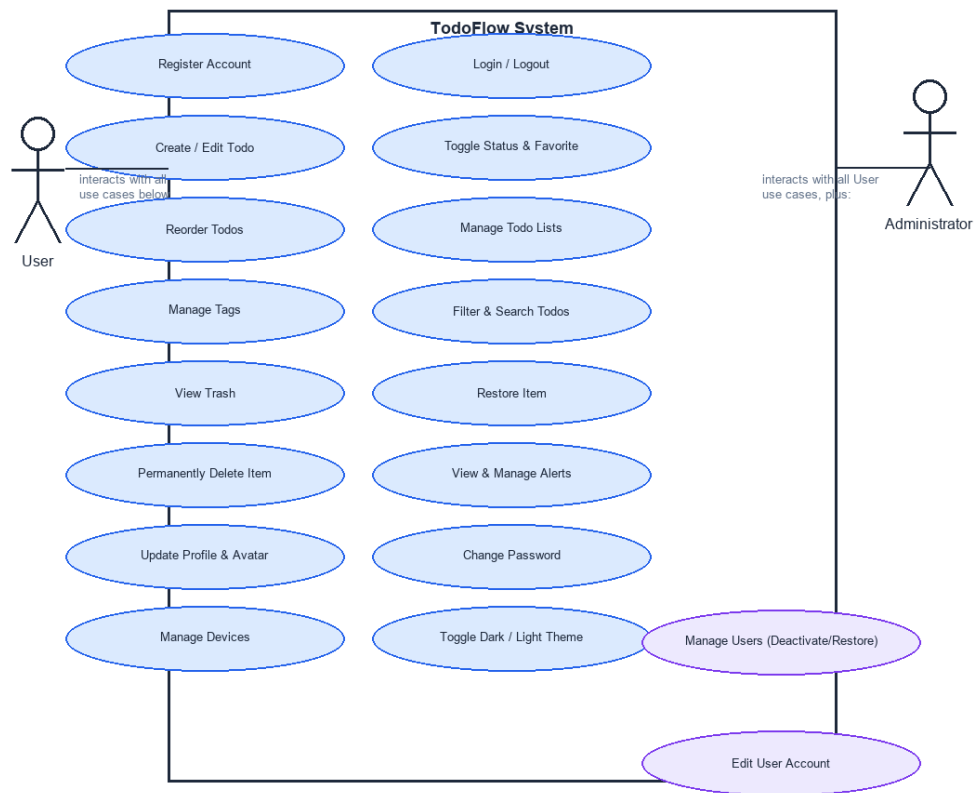


Figure 3.4: Use Case Diagram for User and Administrator Actors

Fig 4: Use Case Diagram for User and Administrator Actors

3.5.3 ER Diagram

Figure 3.5 shows the entity relationship diagram for TodoFlow's database. A User owns many todo lists, todos, tags, devices, sync logs, and change records; a Todo belongs to one todo list and may carry many tags through the tag_todos pivot table; and a Tag with a null user_id is a system tag, available to every user but owned by none of them.

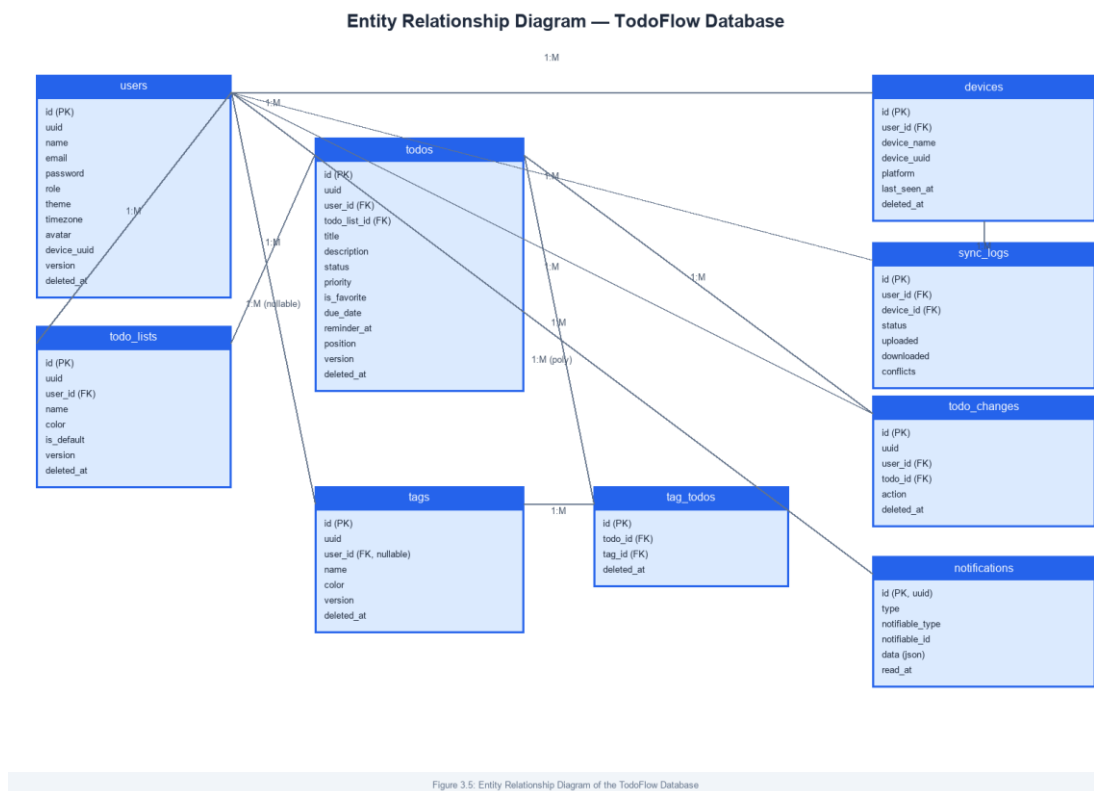


Fig 5: Entity Relationship Diagram of the TodoFlow Database

3.5.4 System Architecture Diagram

In addition to the notations requested by the department's guide, Figure 3.6 documents TodoFlow's layered software architecture, since this project places particular emphasis on separating presentation, business logic, and data concerns (discussed fully in section 4.2.1). Requests flow down from the Bootstrap/Livewire presentation layer through Livewire components that validate input, into a dedicated service layer that holds all business logic and authorisation checks, down to Eloquent models and policies, and finally to the relational database. A cross-cutting error-handling trait and a dedicated log channel wrap every action across this stack.

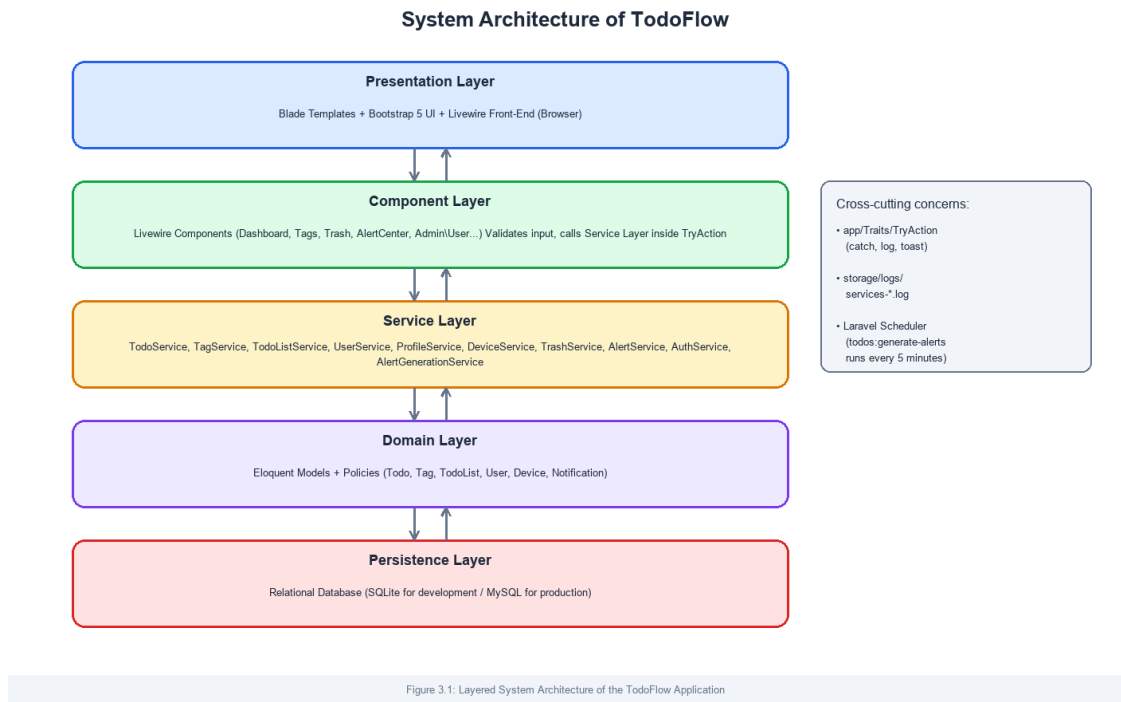


Fig 6: Layered System Architecture of the TodoFlow Application

3.5.5 Database Schema

Figure 3.7 lists every table in TodoFlow's database schema together with its columns, corresponding to the entities shown in the ER diagram in Figure 3.5. Every user-owned table carries a `deleted_at` column supporting soft deletion, and a `version/client_updated_at` pair reserved for a future multi-device synchronisation layer (see section 5.3.3).

Database Schema Overview — TodoFlow

users	id, uuid, name, email, password, avatar, timezone, theme, role, device_uuid, is_synced, version, client_updated_at, last_synced_at, created_at, updated_at, deleted_at
todo_lists	id, uuid, user_id, name, color, is_default, version, client_updated_at, created_at, updated_at, deleted_at
todos	id, uuid, user_id, todo_list_id, title, description, status, priority, is_favorite, due_date, reminder_at, started_at, completed_at, archived_at, position, is_synced, version, client_updated_at, last_synced_at, device_uuid, deleted_at, client_created_at, updated_at, deleted_at
tags	id, uuid, user_id (nullable), name, color, version, client_updated_at, created_at, updated_at, deleted_at
tag_todos	id, todo_id, tag_id, created_at, updated_at, deleted_at
devices	id, user_id, device_name, device_uuid, platform, last_seen_at, created_at, updated_at, deleted_at
sync_logs	id, user_id, device_id, status, uploaded, downloaded, conflicts, created_at, updated_at
todo_changes	id, uuid, user_id, todo_id, action, created_at, updated_at, deleted_at
notifications	id (uuid), type, notifiable_type, notifiable_id, data (json), read_at, created_at, updated_at

Figure 3.6: Database Schema Showing Tables and Columns

Fig 7: Database Schema Showing Tables and Columns

CHAPTER FOUR: TESTING AND IMPLEMENTATION

4.1 Introduction

This chapter reports what we found while testing TodoFlow and what it took to get the system into a working, deployable state. Testing was not a single phase carried out at the end of development; because the project followed the iterative model described in section 3.2.1, alpha testing happened continuously as each module was built, with a dedicated round of manual and automated testing at the end of every feature before it was considered complete. The system has been implemented and demonstrated on a local development server, with the screenshots in this chapter and in Appendix A.1 taken directly from a running instance of the application.

4.2 System Testing

TodoFlow was tested at two levels. Automated feature tests, written using the Pest testing framework, exercise every Livewire component's public methods directly — creating, editing, deleting, restoring, and authorising access to todos, lists, tags, devices, and user accounts — without needing a browser. By the end of development, the automated suite contained 119 tests covering authentication, cross-user authorisation (verifying that one user can never read, modify, or delete another user's data), soft-delete and restore behaviour, the alert-generation command, and the error-handling behaviour described in section 4.2.1. Running the full suite takes under fifteen seconds and was re-run after every significant change throughout development. Manual testing, using the Playwright browser-automation tool driving a real Chromium browser against the running application, complemented this by checking things an automated feature test cannot: whether a modal actually renders correctly on screen, whether a dropdown menu stays open when it should, and whether the interface looks right in both light and dark themes.

4.2.1 Alpha Testing

The most significant coding challenge encountered during development was not a single bug but an architectural one: as the system grew, business logic — database queries, authorisation checks, validation of business rules such as "the default list cannot be deleted" — became scattered directly inside Livewire components, making the components harder to test in isolation and creating a real

risk that an unhandled exception in any single action (a record deleted by another request a moment earlier, for instance) would crash the whole interface for that user. The solution was a project-wide refactor: all business logic was moved into nine dedicated service classes (one each for todos, todo lists, tags, devices, user profiles, the trash system, administrator user management, alerts, and authentication), and every Livewire action method was rewritten to do nothing but validate its input and call the relevant service inside a shared `TryAction` trait. That trait catches any exception a service throws, logs it to a dedicated services log channel separate from Laravel's default log, and shows the user a clear error message instead of an unhandled crash. This refactor touched ten Livewire components and was verified safe by re-running the full automated test suite after each component was migrated, one at a time, rather than all at once.

A second notable bug was caught not by an automated test but by re-reading our own code change carefully: an early version of the refactor described above accidentally left a success toast ("List renamed.") and form-reset logic running even when the underlying save operation had failed, because that code sat just outside the error-handling closure rather than inside it. The existing tests did not catch this because none of them specifically asserted that a success message must not appear on failure — a useful lesson that was carried into how later tests were written. A third bug, found only through manual browser testing rather than automated tests, was that Livewire's DOM-patching mechanism was silently stripping a Bootstrap dropdown's "open" state every time a button inside that dropdown triggered a server round-trip, closing the alert-centre dropdown the instant a user clicked "Mark all read" inside it. This was fixed using Livewire's `wire:ignore.self` directive, which tells Livewire not to overwrite an element's own browser-managed attributes while still updating its contents normally — the documented solution for exactly this class of conflict between server-rendered markup and client-side JavaScript libraries.

Most other coding errors encountered were more ordinary: Livewire validation rules that needed to exclude the record currently being edited (using Laravel's `Rule::unique()->ignore()`), Eloquent relationship queries that needed explicit ownership scoping to prevent one user from acting on another user's record by guessing its numeric ID, and CSS specificity conflicts between Bootstrap's default styles and TodoFlow's custom theme variables. Each of these was found and

resolved through a combination of the automated test suite, manual browser testing, and, where relevant, consulting the official Laravel and Livewire documentation.

4.2.2 Beta Testing

Once the core system was functional, it was demonstrated informally to classmates and reviewed against feedback gathered during development discussions. Several suggestions were adopted directly: that every list, table, and page in the application should be responsive on a mobile screen, not only desktop, which led to a dedicated responsive-design pass across the dashboard, tags, trash, and admin-users screens; that destructive actions should not require a confirmation dialog for routine, recoverable deletions (since everything is already soft-deleted into a Trash), but should require one for the single genuinely irreversible action in the system, permanently deleting an item from the Trash; and that every submit button in the system should show a loading indicator while its request is in flight, so a user is never left wondering whether a click registered. One suggestion — building real browser or email push notifications for reminders, rather than only the in-app alert centre — was not implemented in this iteration due to the additional infrastructure (a queue worker, push credentials) it would require, but the alert system was deliberately architected so that adding it later is a small, contained change rather than a redesign; this is carried forward as a recommendation in section 5.3.3.

4.3 System Implementation

ToDoFlow has been implemented and is fully operational on a local development server, accessible through any modern web browser at the application's local URL. Static assets (CSS and JavaScript) are compiled with Vite and served alongside the PHP application; the database is created and populated through Laravel's migration and seeder system, which also seeds the fixed set of system-level tags described in section 3.3.1. For a full production rollout — for example, hosting the system for the Computer Science Department to use department-wide — the following steps would be required: provisioning a server or shared-hosting account running PHP 8.4 and MySQL 8; running the existing database migrations against that server; configuring a queue worker and the Laravel scheduler (via a cron entry) so the alert-generation command continues to run automatically; and a short walkthrough session for the system administrator covering the admin panel's user-

management screen. No specialised end-user training is required beyond the walkthrough a new user gets from the interface itself, since every destructive action already prompts confirmation only where genuinely necessary, as described in section 4.2.2.

The figures below show the completed system in use. Additional screenshots are provided in Appendix A.1.

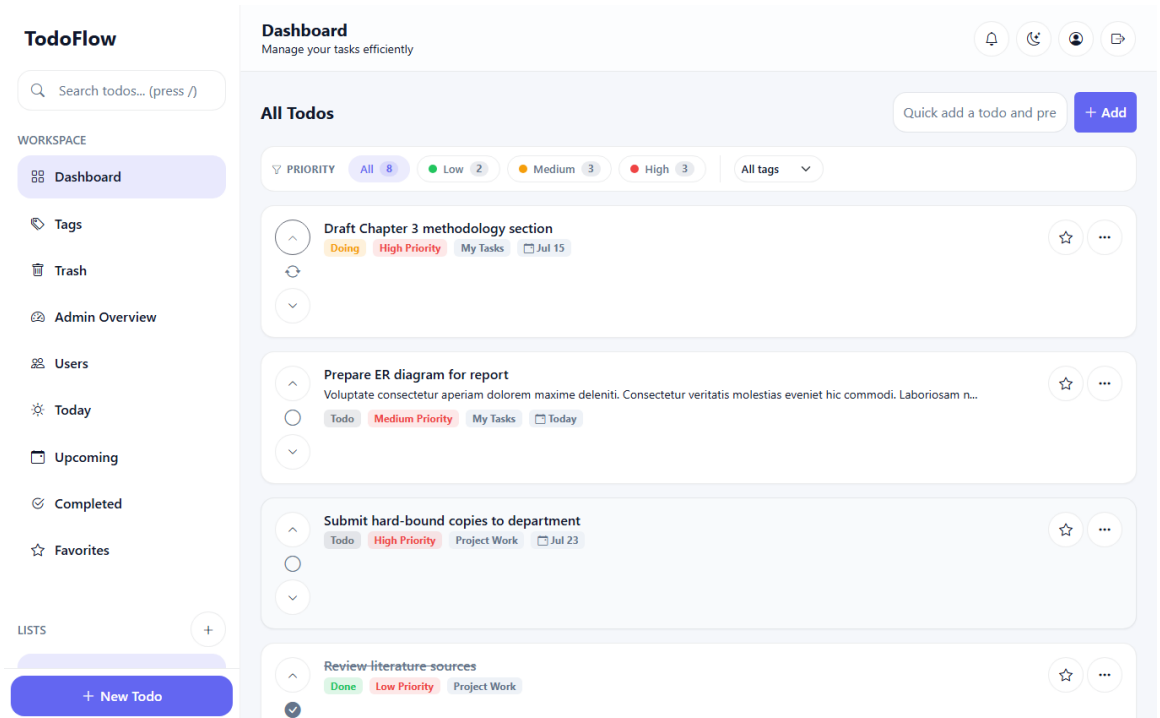


Fig 8: TodoFlow Dashboard — Todo List View with Filters

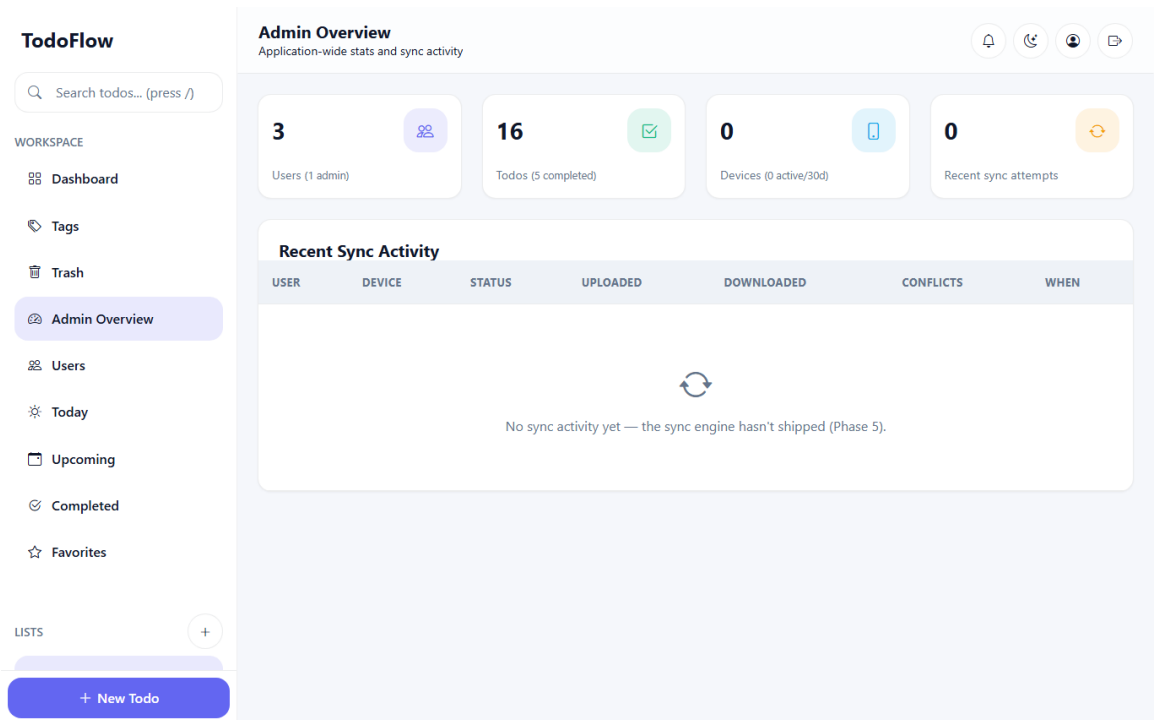


Fig 9: Administrator Overview Screen

CHAPTER FIVE: SUMMARY AND RECOMMENDATIONS

5.1 Introduction

This final chapter summarises the work carried out in this project and sets out our recommendations for the people most likely to build on it: future HND project groups extending or maintaining TodoFlow, end users adopting it for their own task management, and researchers interested in the questions this project raised but did not fully answer.

5.2 Summary

This project set out to design, develop, and test a web-based task management system suited to the connectivity and affordability realities of students and staff at Tamale Technical University. We reviewed three existing task management products — Todoist, Microsoft To Do, and Google Tasks — and identified a consistent gap between what they offer and what a Ghanaian tertiary-institution user actually needs: a free, resilient, shareable system with no paywall between a user and the features that matter most. Guided by that gap and by published literature on offline-first design and mobile internet adoption in Sub-Saharan Africa, we built TodoFlow using Laravel 13, Livewire 4, and Bootstrap 5, implementing full todo, list, and tag management; a recoverable trash system; an automated in-app alert centre; a user profile area; and an administrative panel for user account management. Over the course of development we adopted a layered service architecture with a shared error-handling trait to keep the system stable under unexpected failures, and verified the result with an automated suite of 119 tests alongside manual browser-based testing. The completed system meets every objective set out in section 1.3 and is fully operational on a local development server, ready for the deployment steps described in section 4.3.

5.3 Recommendations

5.3.1 Recommendations for Future Developers

- Complete the multi-device synchronisation layer the database schema already anticipates: every core table carries a version, `client_updated_at`, and (for todos) `device_uuid` column that a future sync engine can use for conflict resolution, but no sync API currently exists.

- Extend the alert system's TodoAlert notification class to deliver through additional Laravel notification channels (mail, browser push) alongside the existing in-app database channel, which was deliberately structured to make this a small addition rather than a redesign.
- Package the application for native mobile deployment using the NativePHP scaffolding already present in the project but not yet wired into the main application.
- Add a queue worker for any future notification channel that should not block the request/response cycle, since the current database-notification channel is synchronous by design.
- Extend the administrator panel with department- or class-level grouping of users, which would let an institution the size of Tamale Technical University manage accounts by cohort rather than as one flat list.

5.3.2 Recommendations for End Users

- Set a due date or reminder on any todo with a real deadline; the alert centre only surfaces todos that have one of these set.
- Use system tags (visible with a shield icon on the Tags page) for common categories instead of recreating your own personal tag with the same name, since system tags are shared and cannot be accidentally duplicated.
- Check the Trash periodically before assuming a deleted item is gone for good — every deletion in TodoFlow is recoverable except the explicit "Delete forever" action.
- A stable internet connection is not required for everyday use once the page has loaded, but an initial connection is currently required to log in and load the application; this is the boundary noted in section 1.5 and section 5.3.3.

5.3.3 Recommendations for Future Research

This project's central unanswered question is how much of TodoFlow's design already anticipates full offline capability and how much genuine engineering remains before a user could create and edit todos entirely without a network connection and have them reconcile automatically once connectivity returns. A focused piece of future research could measure, empirically, how students at Tamale Technical University actually use a task manager under real network conditions on campus, and

use that data to decide between competing conflict-resolution strategies (last-write-wins versus field-level merging) for the synchronisation layer recommended in section 5.3.1 — a decision this project deliberately left open rather than guessing at without usage data.

REFERENCES

The following sources were consulted in the preparation of this project work and are cited in the American Psychological Association (APA) format.

- Nurzi, F. A., & Wahab, K. A. (2022). Web-based student task management system. *Engineering, Agriculture, Science and Technology Journal (EAST-J)*.
- Pothineni, S. H. (2024). Offline-first mobile architecture: Enhancing usability and resilience in mobile systems. *Journal of Artificial Intelligence General Science (JAIGS)*, 7(1), 320–326.
- Shanahan, M., & Bahia, K. (2024). The state of mobile internet connectivity 2024. GSMA.

APPENDIX

A.1 Screen Shots

The following pages present full-screen captures of TodoFlow's main screens, taken from the running application.

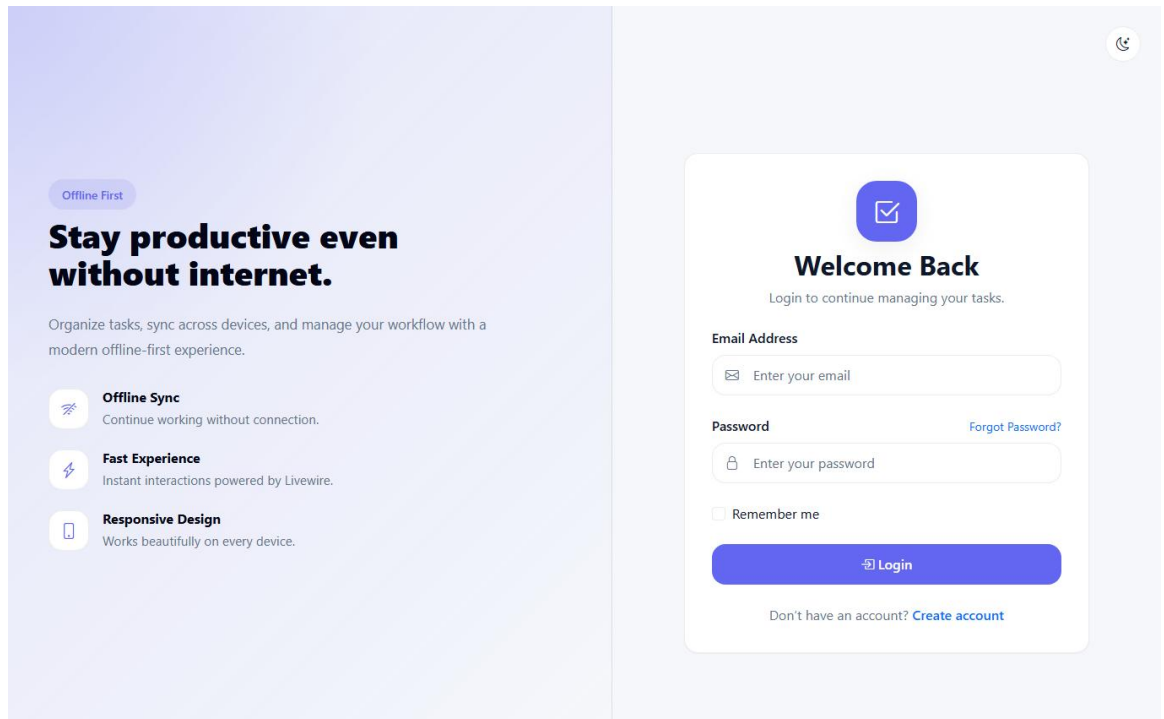


Fig 10: Login Screen

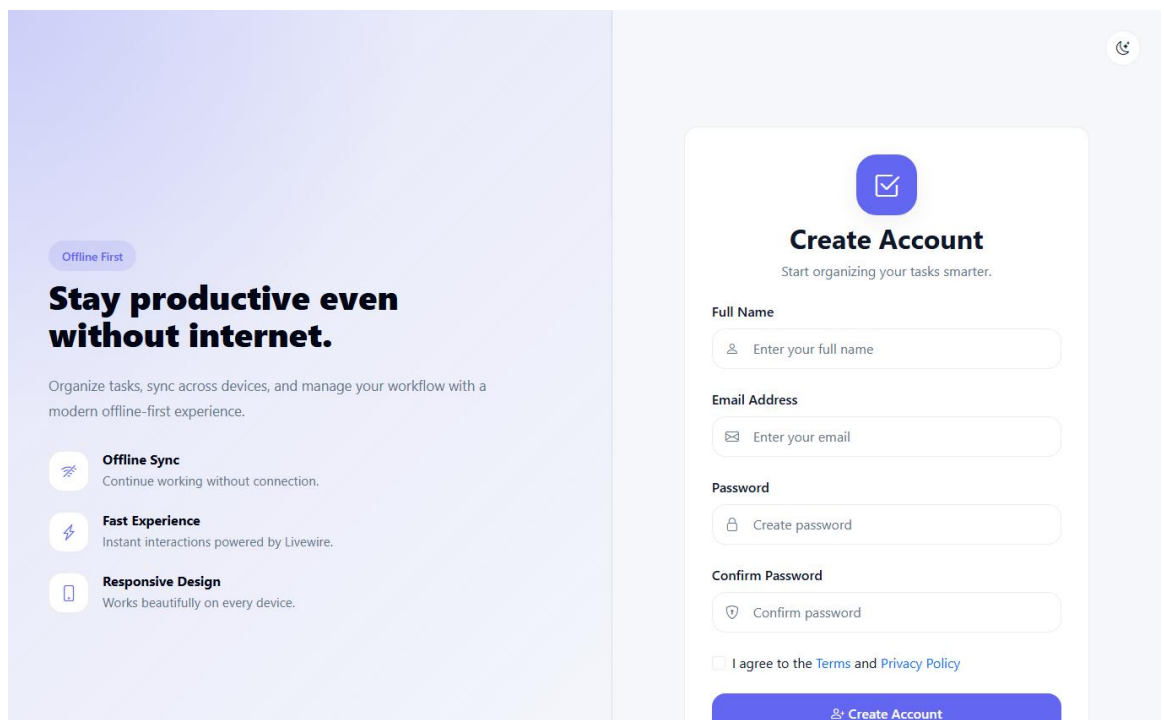


Fig 11: Registration Screen

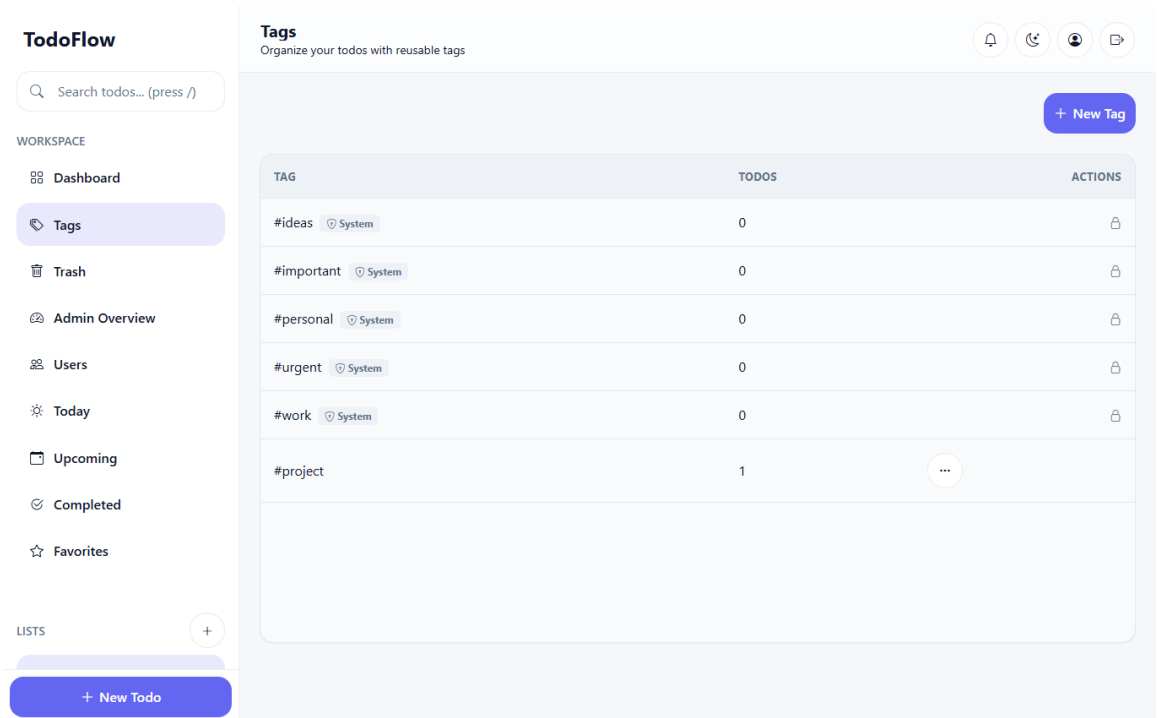


Fig 12: Tag Management Screen, Showing System and Personal Tags

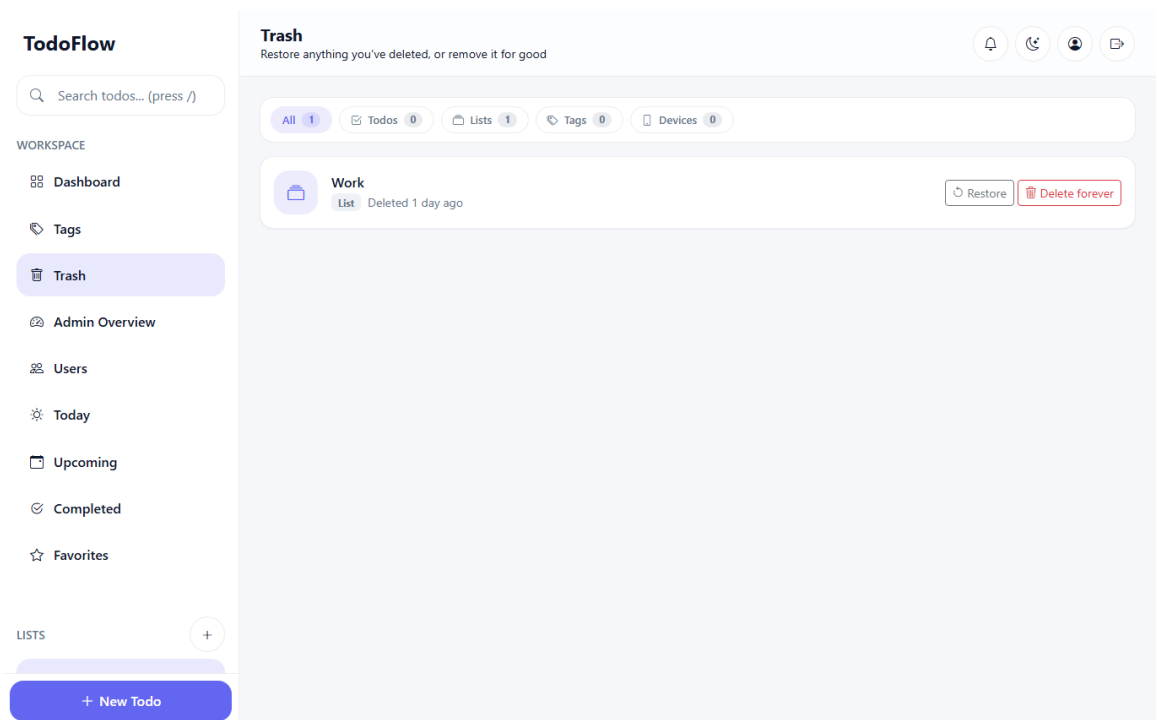


Fig 13: Trash Screen with Model-Type Filter

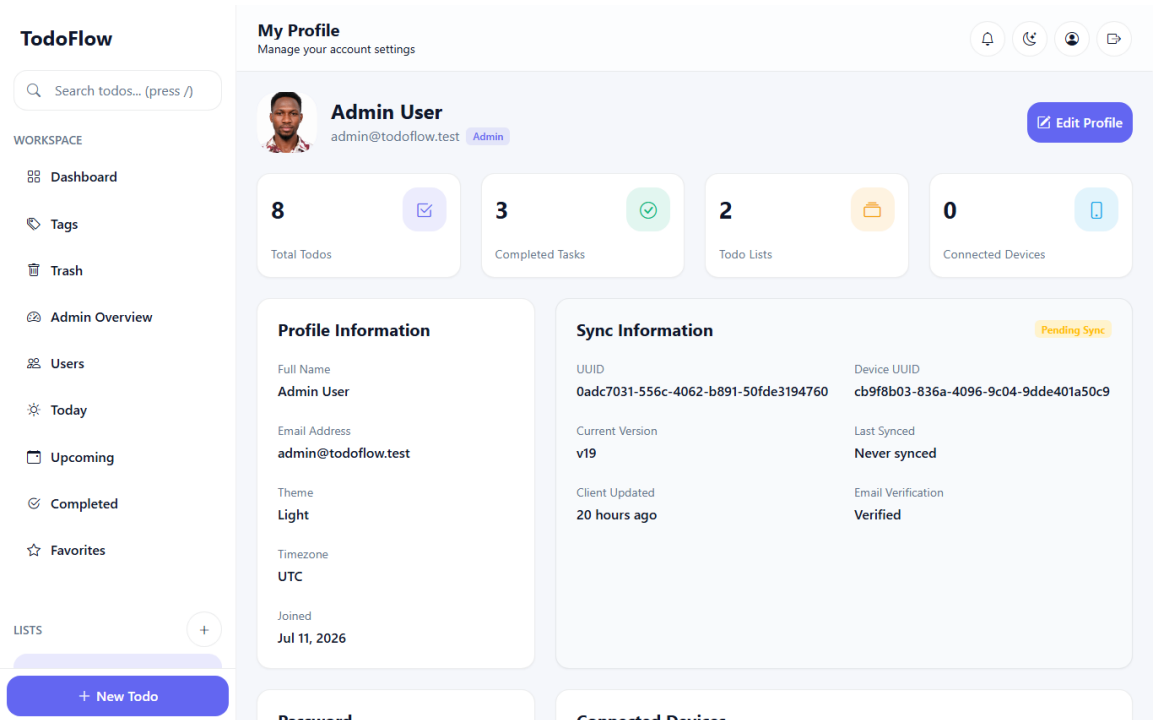


Fig 14: User Profile Screen

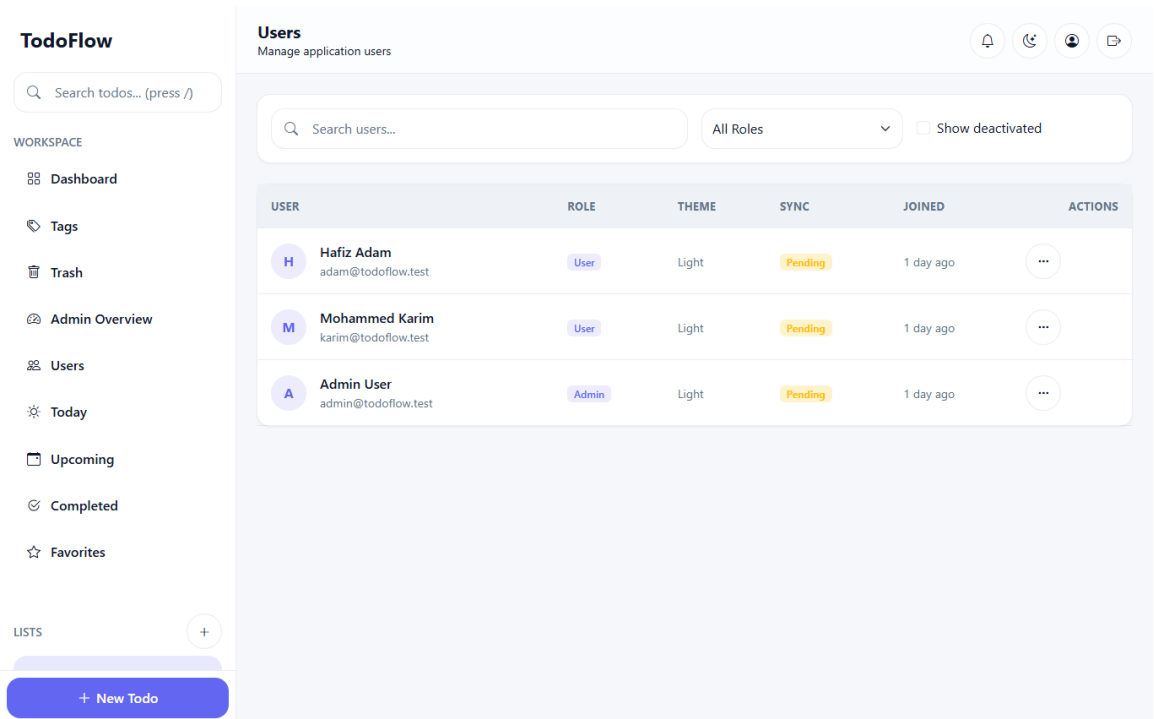


Fig 15: Administrator User Management Screen